WFH EMPLOYEE ATTENDANCE API

Technical Documentation

Project	Employee Work From Home Attendance Backend
Framework	NestJS 12 · TypeScript · Prisma 7
Database	MySQL
Authentication	JWT Bearer Token · Role-based access
Document version	1.0 · 2 November 2026

This document describes the architecture, data model, business rules, API capabilities, setup process, and implementation notes of the supplied backend source code.

1. Executive Summary

The WFH Employee Attendance API is a REST backend for recording employee work-from-home attendance. It separates authentication accounts from employee master data, restricts operations by role, stores daily attendance records, and protects access to attendance evidence photos.

The supplied implementation supports two roles: ADMIN (HRD) and EMPLOYEE. An employee submits one attendance record per Jakarta calendar day with a required photo. Admin manages employee data and monitors attendance records without editing or deleting attendance through the API.

2. Technology Stack

Component	Implementation
Runtime	Node.js (development environment: 24.14.0)
Language	TypeScript 6
Framework	NestJS 12
ORM / client	Prisma 7.10
Database	MySQL, using Prisma MariaDB adapter
Authentication	Passport JWT and @nestjs/jwt
Password security	bcrypt
Validation	class-validator and class-transformer
File upload	Multer / @nestjs/platform-express

3. Architecture Overview

The application follows NestJS modular architecture. Controllers expose HTTP routes and delegate business logic to services. PrismaService provides database access. DTO classes validate request data before it reaches service methods.

Module	Responsibility
AuthModule	Login, JWT generation, JWT strategy, authentication guard and role guard.
EmployeesModule	Admin-only employee CRUD and account activation/deactivation.
AttendanceModule	Employee attendance submission/history, Admin monitoring, and protected photo streaming.
PrismaModule	Shared Prisma Client connection using PrismaMariaDb adapter.
AppModule	Loads global configuration and registers application modules.

Request flow: HTTP request → global ValidationPipe (where DTO is used) → JWT guard → role guard → controller → service → Prisma Client → MySQL. For photo retrieval, the service checks record ownership and streams the image from local storage.

4. Roles and Authorization

Role	Capabilities
ADMIN	Login; create/list/view/update employees; activate/deactivate employees; view all attendance; filter/search/paginate attendance; view any attendance photo.
EMPLOYEE	Login; submit own attendance; view own attendance history; view own attendance photo only.

Protected endpoints use a Bearer JWT. Role metadata is declared using @Roles() and enforced by RolesGuard. Attendance photo access has an additional ownership check: an EMPLOYEE can only retrieve a photo when the attendance record's employee.userId matches the authenticated user ID.

5. Database Design

5.1 User

Stores authentication and authorization data: id (primary key), email (unique), password (bcrypt hash), role (ADMIN or EMPLOYEE), isActive, and createdAt. A User may have zero or one Employee record. Admin accounts do not require an Employee record.

5.2 Employee

Stores employee master data: id, userId (unique foreign key), employeeCode (unique), fullName, department, position, and phone. Each Employee is linked to one User.

5.3 Attendance

Stores employeeId (foreign key), attendanceDate (DATE), attendanceTime (DATETIME(3)), photoPath, and createdAt. The composite unique constraint on (employeeId, attendanceDate) enforces one attendance per employee per date.

Relationship	Cardinality / constraint
User → Employee	One-to-zero-or-one; Employee.userId is unique.
Employee → Attendance	One-to-many.
Employee + attendanceDate	Unique composite key; prevents duplicate daily submissions.

6. Business Rules

- An employee can submit attendance once per Jakarta calendar date.
- The backend generates the attendance timestamp; client-supplied employee ID/date/time are not used by the submission endpoint.
- The date component is derived using the Asia/Jakarta timezone.
- A photo is mandatory. Accepted extension/MIME combinations are JPG/JPEG or PNG, with a maximum size of 5 MB.
- The database stores a relative photo path; the image file itself is stored on disk under uploads/attendance/.
- Employee accounts are deactivated instead of deleted, preserving historical attendance data.
- Admin attendance monitoring is read-only; no attendance update/delete route is implemented.

7. API Reference

Base URL for local development: http://localhost:3000. Protected routes require Authorization: Bearer .

Method	Endpoint	Access	Description
POST	/auth/login	Public	Authenticate and receive JWT.
GET	/auth/profile	Authenticated	Return JWT user profile.
POST	/employees	ADMIN	Create employee and login account.
GET	/employees	ADMIN	List employees.
GET	/employees/:id	ADMIN	Get employee detail.
PATCH	/employees/:id	ADMIN	Update employee master data.
PATCH	/employees/:id/deactivate	ADMIN	Deactivate employee account.
PATCH	/employees/:id/activate	ADMIN	Activate employee account.
POST	/attendance	EMPLOYEE	Submit attendance with photo.
GET	/attendance/me	EMPLOYEE	Get own attendance history.
GET	/attendance	ADMIN	Monitor attendance with filters and pagination.
GET	/attendance/:id/photo	ADMIN / owning EMPLOYEE	Stream protected attendance photo.

7.1 Attendance monitoring query parameters

Parameter	Type	Default / limit	Purpose
date	ISO date	Optional	Exact attendance date filter, e.g. 2026-11-02.
employeeId	Integer	Optional, min 1	Filter by Employee table ID.
search	String	Optional	Search employee fullName or employeeCode.
page	Integer	1, min 1	Page number.
limit	Integer	10, range 1–100	Records per page.

7.2 Attendance submission

POST /attendance uses multipart/form-data with a required file field named photo. Employee identity is obtained from the JWT. Do not send employeeId, attendanceDate, or attendanceTime. A duplicate submission returns 409 Conflict.

7.3 Photo retrieval

GET /attendance/:id/photo returns an image stream (image/jpeg or image/png), not a JSON object. Admin can access any existing photo. An Employee can access only their own photo. Missing record/file returns 404; cross-employee access returns 403.

8. Setup and Configuration

8.1 Prerequisites

- Node.js and npm (the supplied environment used Node.js 24.14.0).
- A running MySQL server and a database user with privileges to create/alter tables.
- Postman or another HTTP client for testing.

8.2 Install dependencies

```
npm install
```

8.3 Environment variables

```
DATABASE_URL="mysql://root:your_password@localhost:3306/absensi_wfh"
DATABASE_HOST="localhost"
DATABASE_PORT="3306"
DATABASE_USER="root"
DATABASE_PASSWORD="your_password"
DATABASE_NAME="absensi_wfh"
JWT_SECRET="replace_with_a_long_random_secret"
ADMIN_EMAIL="admin@absensiwfh.com"
ADMIN_PASSWORD="replace_with_a_strong_admin_password"
PORT="3000"
```

DATABASE_URL is used by the Prisma CLI configuration. The DATABASE_HOST/PORT/USER/PASSWORD/NAME variables are used by the runtime PrismaMariaDb adapter. PORT is optional; the application defaults to 3000.

8.4 Database setup

Create the database (example: CREATE DATABASE absensi_wfh;), then run:

```
npx prisma generate
npx prisma migrate deploy
npm run db:seed:admin
```

The Admin seed reads ADMIN_EMAIL and ADMIN_PASSWORD. It hashes the password with bcrypt. When an account with that email already exists, the current upsert branch restores ADMIN role and active status but does not replace the existing password hash.

8.5 Start and build

```
npm run start:dev
npm run build
npm run start:prod (after build)
```

9. File Upload and Storage

- Multer writes uploaded files to the project working directory at uploads/attendance/.
- Filenames are generated using UUIDs and retain the lowercased original extension.
- The database stores paths such as uploads/attendance/.jpg.
- The photo endpoint resolves the path, verifies it remains inside the attendance upload directory, checks file existence, and streams it.
- The current filter checks declared MIME type and filename extension; it does not verify file magic bytes/content signatures.

For production, use persistent/shared object storage when deploying multiple application instances. Add file-signature inspection and malware scanning where required.

10. Validation and Error Handling

HTTP status	Meaning / common trigger
200	Successful read or update.
201	Successful create or attendance submission.
400	DTO validation failure, missing photo, or unsupported file type.
401	Missing/invalid/expired JWT or invalid login credentials.
403	Role restriction or Employee attempting to access another employee's photo.
404	Employee, attendance record, or photo file not found.
409	Unique constraint conflict: duplicate email/code or duplicate daily attendance.
413	Upload exceeds configured 5 MB Multer limit.
500	Unexpected server-side error.

Global ValidationPipe is configured with whitelist=true, forbidNonWhitelisted=true, and transform=true. This means undeclared request properties are rejected and numeric query values can be transformed before validation.

11. Postman Test Plan

- Admin login and profile retrieval.
- Admin creates, lists, retrieves, updates, deactivates, and reactivates an employee.
- Employee login and profile retrieval.
- Employee submits a valid photo and receives a successful response.
- Reject missing photo, invalid extension/MIME, and files larger than 5 MB.
- Reject a second attendance submission on the same Jakarta date.
- Employee retrieves own attendance history.
- Admin retrieves attendance with date, employeeId, search, page, and limit filters.
- Employee cannot access Admin-only employee or monitoring routes.
- Admin opens an attendance photo; owning Employee opens own photo; another Employee receives 403.
- A non-existing attendance/photo ID returns 404.

12. Implementation Notes and Production Considerations

- JWT expiration is hardcoded to one day in AuthModule (signOptions.expiresIn: '1d').
- JwtStrategy validates token signature and expiration but does not query the database for current isActive status on each request. Immediate revocation after deactivation would require active-account revalidation or a token revocation strategy.
- Upload validation currently trusts the declared MIME type plus extension. Add file-signature inspection for stronger content validation.
- Local disk uploads are appropriate for development/single-instance demo use; production deployments should use persistent shared storage.

- Use HTTPS, strong secrets, least-privilege DB credentials, secure backups, and centralized logs in production.
- The supplied source tree uses src/attendances (plural) as the module folder while the public route prefix is /attendance (singular).

This documentation was prepared from the supplied project archive. Request/response examples are representative; generated IDs, timestamps, tokens, and uploaded filenames will differ at runtime.